Amazon Kinesis

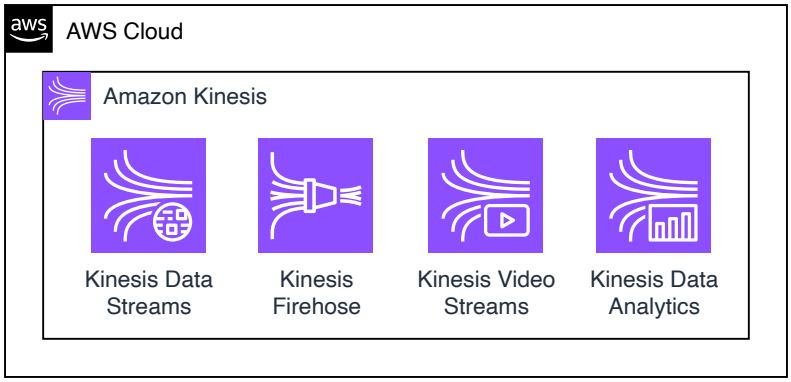
Explore the core Amazon Kinesis services that support real-time streaming data processing and analytics. Understand how to ingest, process, and analyze continuous data streams using Kinesis Data Streams, Firehose, Video Streams, and Data Analytics. Learn the scaling, monitoring, and troubleshooting techniques essential for building efficient, event-driven AWS applications with real-time insights.

In today's data-driven applications, we often need to process continuous data streams, like sensor data from IoT devices, website clickstreams, or logs from distributed systems. **Amazon Kinesis** is a suite of services that Amazon Web Services (AWS) offers for collecting, processing, and analyzing real-time streaming data. It's designed to handle high volumes of data from various sources, such as website clicks, social media feeds, sensor data, financial transactions, etc.

Amazon Kinesis services

Amazon Kinesis is serverless and allows us to define how we want to process the data streams and integrate the processed data with other AWS services fo analysis. Here are four primary services under the Amazon Kinesis that we'll cover in this lesson:

Ask



Services offered by Amazon Kinesis

- **Kinesis Data Streams** allows us to build real-time applications that consume high-throughput data with millisecond latency. We write data to a stream using AWS SDKs or the AWS CLI, then process that data using consumers like Lambda functions, EC2 applications, or custom code. The illustration below shows a stream produced by web servers called stream producers, which is ingested by Kinesis Data Streams to process them and send them to EC2 or Lambda, the stream consumers in this case:

Kinesis Data Streams use case

- **Kinesis Data Firehose** simplifies the delivery of real-time streams to destinations like Amazon S3, Redshift, OpenSearch, or even custom HTTP endpoints. It handles buffering, batching, retrying, and optional

Ask

transformation using AWS Lambda. The illustration below shows Kinesis Data Firehose using processed data packages from Kinesis Data Streams and delivering them to the warehouse destination service, like S3. Consumer applications in EC2 instances then retrieve packages from the warehouses for further processing or analysis.

Kinesis Data Firehose use case

- **Kinesis Video Streams** is a specialized service for ingesting and processing video data from connected devices such as surveillance cameras, drones, smartphones, or body-worn cameras. It enables real-time video streaming into AWS for storage, playback, and machine learning-based analysis. The illustration below shows video footage being ingested from edge devices into Kinesis Video Streams, where it can be processed in real-time for use cases like anomaly detection, security monitoring, or search and rescue. Downstream consumers can access the video data for further analytics or integration with AI services.

Ask

Kinesis Video Streams use case

- **Kinesis Data Analytics** enables us to process and analyze streaming data using standard SQL. It allows us to create and run powerful SQL code against streaming sources for time series analytics, real-time dashboards, and custom metrics. This is useful for real-time dashboards, anomaly detection, or filtering noise from raw events. To understand the working of Kinesis Data Analytics, let's divide it into three major components:
    - **Application:** The basic resource in Kinesis Data Analytics is an application. The application is written down in SQL using an interactive editor and tested with live-streaming data.
    - **Input:** An application can continuously read and process streaming data from Amazon Kinesis Data Streams or Amazon Data Firehose. We can enrich the input data by using a reference table, which gets data as objects from the S3 bucket.
    - **Output:** The application executes the SQL queries on the data from the source and stores them in the configured destination, either a Firehose delivery stream or a Kinesis data stream.

The illustration below will help us understand the Amazon Kinesis Data Analytics workflow.

Ask

Workflow of Amazon Kinesis Data Analytics

Here, Kinesis Data Analytics creates these continuously updated streams within the application to perform SQL queries and store intermediate results. We can also partition these streams into multiple in-application streams to improve the throughput.

Real-time ingestion and processing patterns

A typical Kinesis data pipeline starts with **producers**, applications or devices that continuously send data records to a Kinesis Data Stream. Producers can include web or mobile applications, IoT sensors, server logs, or third-party services that emit continuous data. These producers use the AWS SDK, the AWS CLI, or Kinesis Agent to publish records to a stream with an associated partition key.

Here's a sample code in Python that uses Boto3 to simulate a producer that publishes records to a stream:

Ask

main.py Python

```
import boto3
import json
```

```
kinesis = boto3.client('kinesis', region_name='us-east-1')  # specify

stream_name = "<stream_name>"
partition_key_prefix = "user-"

for i in range(10):
    data = {
        "event": "purchase",
        "user_id": f"user-{i}",
        "amount": round(20 + i * 1.5, 2)
```

Publish records to the stream

In the code above, we initialize the Kinesis client on **line 4,** and, on **lines 16–20**, we send JSON-formatted records to the stream with a user-based partition key.

Once data lands in the stream, it becomes available for real-time processing by consumers. Consumers are applications that retrieve records from Kinesis to perform actions such as enrichment, filtering, alerting, or storing in a database. The following are common ways to process streaming data using Kinesis consumers:

- Trigger AWS Lambda functions to handle streaming data serverlessly, ideal for lightweight processing or fan-out scenarios.
- Use the **Kinesis Client Library (KCL)** or Kinesis Data Streams API for more sophisticated, stateful consumers.
- Integrate with Amazon Kinesis Data Analytics to run SQL queries for stream transformation.

Here's a sample consumer Lambda code that processes the records in the stream and prints it:

Ask

main.py Python

```
def lambda_handler(event, context):
```

```
    for record in event['Records']:
        payload = base64.b64decode(record['kinesis']['data'])
        print(json.loads(payload))
```

Consumer Lambda code

These processing patterns are foundational for real-time dashboards, fraud detection systems, telemetry ingestion, and clickstream analytics.

Scaling and performance

Amazon Kinesis Data Streams is designed to scale horizontally through a construct called **shards**. Each shard provides a baseline throughput of 1 MB per second for writes (or 1,000 records per second, whichever limit is hit first) and 2 MB per second for reads. If an application demands more throughput, we can scale by adding more shards or splitting existing ones using the `UpdateShardCount` API or AWS Management Console.

Ask

Each record sent to a stream includes a partition key, which determines the sha
that receives the data. This key ensures per-shard ordering, making it essential

use cases that rely on event sequence integrity, such as financial transactions o device telemetry.

> **Note:** To serve multiple independent consumer applications without degrading throughput, *enhanced fan-out* can be enabled. This feature assigns each consumer its own read throughput channel (up to 2 MB/sec per consumer, per shard), allowing for highly parallel, low-latency consumption of the same data stream.

Data retention and ordering

Amazon Kinesis Data Streams allows us to retain streaming data for up to 365 days, with a default retention period of 24 hours, configurable to 7 days withou extended retention. This capability supports powerful replay use cases, such a

- Reprocessing with updated business logic or downstream consumers.
- Backfilling data into new analytics platforms.
- Debugging and forensic analysis after an incident.

We can use the `GetShardIterator` and `GetRecords` APIs to replay data by specifying a timestamp or sequence number.

Ordering within Kinesis is guaranteed at the shard level. By using a consistent partition key, related data, such as all logs from a specific device or user sessio is routed to the same shard. This makes Kinesis ideal for event-driven architectures where sequence matters.

Monitoring and troubleshooting

To maintain a healthy Kinesis pipeline, active monitoring and diagnostics are essential. Amazon CloudWatch provides near real-time visibility into data ingestion, consumer lag, and throughput utilization. The illustration below show how Kinesis integrates with CloudWatch:

Ask

Using CloudWatch to monitor Kinesis

The following metrics are useful while working with Kinesis:

- `PutRecord.Success`: Indicates successful writes to the stream.
- `PutRecord.Throttled`: Alerts on write requests being throttled due to shard limits.
- `GetRecords.IteratorAgeMilliseconds`: Measures the age of the last record read by consumers. A rising value signals consumer lag.
- `IncomingBytes` and `OutgoingBytes`: Help monitor shard capacity consumption.

To trace the performance of consumers like Lambda and EC2 instances, and identify bottlenecks, visualize downstream impacts, we can use AWS X-Ray. The illustration below shows how X-Ray identifies bottlenecks at the consumer's end:

Ask

Using X-Ray to trace consumer's performance

We can also implement the following measures to handle errors on the consumer side:

- Implement retry logic with exponential backoff to gracefully handle transient failures.

Ask